Login flow, data model & personalised routing

Phone browser | Next.js server | PostgreSQL (Prisma) | Amazon SES / SNS

Sign-in sequence

1

PHONE → SERVER

Guest enters BAPS ID on /

POST /api/auth/request-otp { bapsId } — body validated with zod.

2

SERVER → POSTGRES

Look up the delegate, mint a code

SELECT users WHERE baps_id → expire old codes → INSERT otp_codes with code_hash = HMAC-SHA256(AUTH_SECRET, userId:code). Clear text is never stored.

10-minute expiry | 3 codes / 10 min | 5 attempts / code

3

SERVER → AWS (SDK v3, HTTPS)

Deliver on the guest's preferred channel

preferred_channel = EMAIL
Amazon SES SendEmail from noreply@…

preferred_channel = SMS
Amazon SNS Publish to the E.164 phone

Response returns only a masked destination — { destination: "up•••@outlook.com" }. OTP_PROVIDER=console prints the code in dev instead.

4

PHONE → SERVER → POSTGRES

Guest types the 6-digit code

POST /api/auth/verify-otp takes the latest unconsumed row, compares the HMAC, increments attempts, then sets consumed_at. A just-verified code stays valid 2 more minutes so mobile autofill double-submits don't error.

5

SERVER → PHONE

Session cookie issued — no session table

HS256 JWT signed with AUTH_SECRET (sub = users.id, 7-day expiry) set as hh_session — httpOnly, Secure, SameSite=Lax.

6

GUARD → HUB

Middleware routes into the hub

src/middleware.ts (Edge) checks only that the cookie *exists* — it holds no secret. Protects /dashboard /schedule /menu /breakouts /profile and bounces a signed-in guest on / straight to /dashboard. Signature is verified later by requireGuest() in Node.

!

Failure paths

Unknown BAPS ID → generic error (no user enumeration). No cookie on a protected route → redirect to /. Valid cookie but the user row is gone → /api/auth/expired clears the cookie and returns to login. /api/auth/logout clears it on demand.

DIAGRAM 1 · CONTINUED

Database schema behind the login flow

Database schema `prisma/schema.prisma`

users — ONE ROW PER DELEGATE

id cuid PK	**baps_id** unique
name	display_name?
seva text	email?
phone? E.164	**gender?** MALE|FEMALE
preferred_channel EMAIL|SMS	region?
breakout_sessions text[] → breakouts.id ({} = unassigned)	
created_at · updated_at	

1 —▶ N — cascade on delete

otp_codes — HASHED ONE-TIME CODES

id cuid PK	**user_id** FK → users.id
code_hash HMAC-SHA256(secret, userId:code)	
channel EMAIL|SMS	destination masked
expires_at	attempts max 5
consumed_at?	created_at
@@index(user_id, created_at)	

CONTENT TABLES READ ON EVERY HUB PAGE

schedule_items
title · description? · location?
kind ARTI|SESSION|BREAKOUT|
MEAL|BREAK|TRAVEL|FREE
audience ALL|BHAIO|BEHNO
starts_at · ends_at (UTC) · sort_order

breakouts
seva? fallback match
title · description?
room · facilitator?
starts_at · ends_at
@@index(seva, starts_at)

meals
schedule_item_id? unique FK
name · note? · location?
starts_at · ends_at

menu_items
meal_id FK → meals.id
course · name · description?
tags[] e.g. "Tentative"
sort_order

No session table — the JWT *is* the session. Content tables are rewritten by `prisma/seed.ts`; `users` is only touched by `scripts/upsert-user.ts`.

Which version of the hub a guest lands on

Everyone shares the same four URLs — there are no per-cohort routes. After `requireGuest()` loads the signed-in `users` row, each server-rendered page filters its content against that row, so the hub a guest sees is assembled per request from **gender**, **seva**, **preferred_channel** and the clock.

`INPUT: users.gender`

Schedule track

forAudience() · /schedule + dashboard

MALE
Sees `ALL` + `BHAIO` items. `BEHNO` filtered out.

FEMALE
Sees `ALL` + `BEHNO` items. `BHAIO` filtered out.

NULL (none on file)
Sees **both** tracks, each labelled Bhaio / Behno. Friday & Saturday are the split days.

`INPUT: users.seva + breakout_sessions[]`

Breakout sessions

getBreakoutsForUser() · /breakouts

breakout_sessions ≠ {}
Exactly the rows whose ids are listed — the explicit per-guest assignment.

breakout_sessions = {} (fallback)
Every `breakouts` row where `seva = user.seva` — Hospitality, Registration, Audio Visual, Kitchen & Prasad, Medical, Logistics, Security…

Sessions are then stamped **Now** / **Up next** / past by `annotateBreakouts()`, same as the schedule.

`INPUT: the clock (EVENT_TZ)`

Menu

annotateMeals() · revealTime() · /menu

Each menu self-reveals 1 hour before its allocated time

`reveals_at = max(previous meal ended, start − 1 h)`

No action by staff, no deploy — the page computes it on every request and re-checks every 60 s.

LOCKED
Before the reveal: time and place only, dishes withheld.

REVEALED
Courses open — e.g. lunch at 12:30 opens at 11:30, once breakfast has ended.

SERVED
After the meal ends it collapses to a "Served" card, still expandable. `MENU_REVEAL_ALL=true` unlocks everything for content audits.

`INPUT: name, channel, region`

Shell & delivery

(hub)/layout.tsx · /profile

Greeting
"Jay Swaminarayan {display_name ?? name}" in the parallax header.

OTP channel
`EMAIL` → SES · `SMS` → SNS. Decided per guest at step 3.

Same nav for all
Dashboard · Schedule · Menu · Breakouts in the frosted bottom bar. Difference is in the data, never the route.

Preview override
`?now=` re-renders any page at another moment when `ALLOW_TIME_TRAVEL=true`; off in production.

Tech stack & how the pieces talk

One deployable Next.js unit · one Postgres
two AWS messaging services

CLIENT

Phone browser

- React 19 client components — login form, day tabs, schedule & menu views
- Frosted bottom nav, scroll reveals, parallax header, portrait guard
- `AutoRefresh` calls `router.refresh()` every 60 s
- Holds only the `hh_session` cookie — httpOnly, unreadable by JS

HTTPS
HTML + JSON
◀ ▶
RSC payload

APPLICATION · NODE 20 · npm start

Next.js 15 App Router

React 19 · TypeScript
Tailwind CSS 4 · lucide-react

middleware.ts (Edge)
Cookie-present check on `/ /dashboard /schedule /menu /breakouts /profile`. No secrets, no DB.

api/auth/* route handlers
`request-otp · verify-otp · logout · expired` — zod-validated bodies.

(hub) server components
Dashboard, Schedule, Menu, Breakouts, Profile — all `force-dynamic`, so every request hits the DB and recomputes "now".

lib/
`auth.ts` (jose JWT + requireGuest) · `otp/` · `schedule.ts` (now/next, reveals) · `time.ts` · `db.ts`

next/font · Playfair + Plus Jakarta self-hosted
@theme tokens: saffron / navy / cream
EVENT_TZ = America/New_York

AWS SDK v3
HTTPS
send-only IAM

MESSAGING · us-east-1

Amazon SES

`SendEmail` (SESv2) for email codes. DKIM-verified sender; sandbox until production access is granted.

Amazon SNS

`Publish` for SMS codes. Needs a registered origination number (toll-free or 10DLC) before US delivery.

Swappable behind `lib/otp/providers/` — `console` in dev, Twilio Verify as a drop-in.

BUILD & OPS

`npm ci && npm run build` → `prisma generate` + `next build`
Secrets via env only · HTTPS termination required (Secure cookie)
Content edits: `seed.ts` → `db:seed:lightsail`, no deploy

Prisma 6 · TCP 5432 · TLS (sslmode=require)
one pooled client from lib/db.ts · connection_limit=10

DATA · THE ONLY PERSISTENT STORE

Amazon Lightsail managed PostgreSQL 18 — `naam-users`

Small bundle · 1 vCPU / 2 GB / 80 GB · single AZ
~800 guests · 1–3 small indexed queries per page

users	otp_codes	schedule_items	meals	menu_items	breakouts

One hub page load, end to end

01 · EDGE
Middleware sees `hh_session` — otherwise redirect to `/`.

02 · NOW
`pageNow()` resolves the moment — real clock or `?now=`.

03 · IDENTITY
`requireGuest()` verifies the JWT and loads the user row once per request.

04 · PERSONALISE
Prisma queries, then `forAudience` (gender), `getBreakoutsForUser` (seva), `annotateMeals` (1-hour reveal).

05 · RENDER
HTML streams to the phone; client bits hydrate and re-run 02–05 every 60 s.